PageGram

Multi-Head N-gram Memory at the Read Cost of a Flat Table

Leon Lehmann

Empero AI Research lehmann@empero.org Correspondence: research@empero.org

Code: github.com/empero-org/pagegram

Abstract

Hashed n-gram tables give a language model billions of parameters that cost memory but almost no compute, and the strongest designs hash every n-gram with several independent heads. On a device, such a table lives in flash, and independent heads turn every token into sixteen scattered reads. PageGram keeps the heads but places all heads of an n-gram order inside one page, so a token reads two pages, as with a flat table. At equal table size the locality costs no quality. In a controlled study PageGram is better than a flat table and than Engram-style multi-head addressing on each of five seeds after 60M training tokens, by 0.067 and 0.142 nats. With twice the training tokens it is level with Engram-style addressing, 0.010 apart, and still below the flat table on average; in a larger model it is level with the flat table within seed noise. The multi-head layouts differ only in how collisions correlate across heads, and a single parameter, the number of slots per head in a page, controls that correlation; four slots is the most reliable setting. Stored at four bits per value, which costs 0.0004 nats, a 20B-parameter PageGram table needs 0.99 flash reads per token with a 1 GiB page cache, against 9.18 for Engram-style addressing, and serves five to eight times as many lookups per second from an NVMe drive.

KEY FINDINGS

- **Eight times fewer flash reads.** A token reads one page per n-gram order, as with a flat table, where Engram-style multi-head addressing needs sixteen scattered reads. On a 20B-parameter table with a 1 GiB page cache, that is 0.99 reads per token against 9.18, and five to eight times as many lookups per second from NVMe.
- **No quality cost, an early lead.** At equal size (200M parameters), PageGram is better than a flat table and than Engram-style addressing on all five seeds after 60M training tokens, by 0.067 and 0.142. With twice the tokens it is level with Engram-style addressing, 0.010 apart, and still below the flat table on average.
- **Correlated collisions are the lever.** Per head, PageGram's collisions are as frequent as Engram-style addressing's; only their correlation across heads differs. Four slots per head per page gives the best mean, and no seed fails at that setting.
- **Deployable as trained.** Snapping the table to 4-bit NF4 costs 0.0004 nats, and training runs within 5% of the speed of a model without a table.

1 Introduction

A hashed n-gram table adds capacity to a language model without adding compute. Every token hashes its recent context, the last two or three tokens, into rows of a large table, and the rows it finds are added to the residual stream. The parameters cost memory and almost no FLOPs, so the table can be far larger than the network that reads it. Recent work scales such tables to tens of billions of parameters (Huang

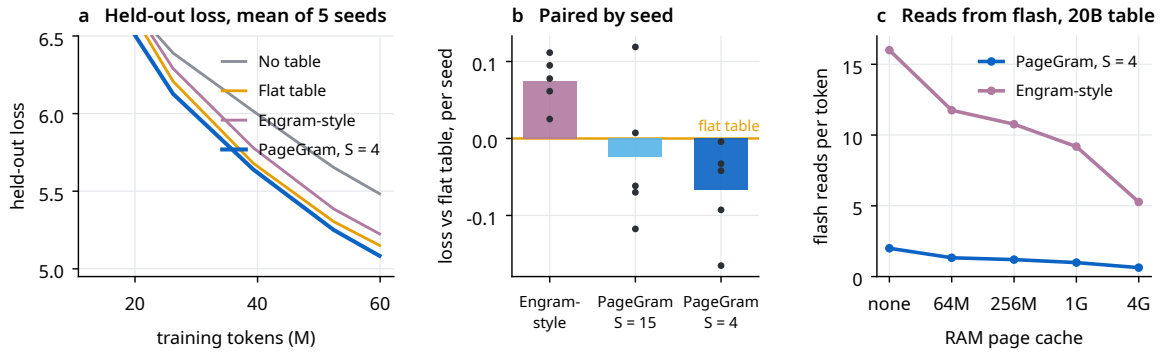


Figure 1 **PageGram matches or beats both standard tables and reads eight times less from flash.** **a** Held-out loss over 60M training tokens of a small decoder with no table, a flat hashed table, Engram-style multi-head addressing and PageGram, all tables of 200M parameters, mean of five seeds. **b** Final loss relative to the flat table, paired by seed; dots are seeds. **c** Random flash reads per token for a 4-bit table of 20B parameters against the size of the RAM page cache, on 20M held-out tokens.

et al., 2025; Cheng et al., 2026; Liu et al., 2026), and they now appear in large production models. The strongest designs share one structure. Each n-gram is hashed by several independent heads, each into its own table, and the head rows are concatenated and decoded by a learned projection (Huang et al., 2025; Cheng et al., 2026). Two n-grams then rarely collide in every head at once.

On a device this structure has a price. A table of this size does not fit in the memory of a phone or a laptop next to the model, so it lives in flash storage, which is read in pages of 4 KiB. Independent heads place a token's rows in independent pages. With eight heads for bigrams and eight for trigrams, every token makes sixteen random reads, and a page cache in RAM helps little, because each cached page holds one useful row. A flat table, with one row per n-gram order, needs only two reads, but it collides completely: two n-grams that share a row are indistinguishable.

PageGram keeps the heads and removes the scatter. Each n-gram order hashes the n-gram to one page, and the heads choose their rows among the slots inside that page (Fig. 2). A token reads two pages, as with a flat table. The rows each head sees are as crowded as before, so we expected locality to cost little. It costs no measurable quality. In a controlled study with tables of 200M parameters, PageGram is better than a flat table and better than Engram-style addressing on each of five seeds after 60M training tokens (Fig. 1). With twice the tokens it is level with Engram-style addressing and still below the flat table on average, and in a larger model it is level with the flat table within seed noise. The early lead comes from how collisions are correlated across heads, which a single parameter, the number of slots per head in a page, controls.

Contributions.

- **PageGram**, a page-local layout for multi-head n-gram tables: two flash reads per token for any number of heads, with a 4-bit page format (Section 3).
- **A controlled comparison** at equal table size, with seeds that share their initialisation and data. After 60M tokens PageGram lowers held-out loss by 0.067 against a flat table and 0.142 against Engram-style addressing, on every seed; with twice the tokens it is level with Engram-style addressing and still below the flat table on average (Section 5).
- **An explanation.** The per-head collision rate is identical across the multi-head layouts, and the correlation of collisions across heads is the lever. Sweeping it gives the best result at four slots per head per page (Section 6).
- **Flash measurements** on real tokens: reads per token, page-cache misses at three table sizes, and lookup throughput from an NVMe drive (Section 8).

- **Code and results.** The harness, the flash simulator and the results of every run are released at `github.com/empero-org/pagegram`.

2 Background

2.1 Hashed n-gram memory

For token x_t with predecessors x_{t-1} and x_{t-2}, a memory with n-gram orders $n \in \{2, 3\}$ hashes the bigram (x_{t-1}, x_t) and the trigram (x_{t-2}, x_{t-1}, x_t) into a table $T \in \mathbb{R}^{R \times w}$. A *flat* table reads one row per order, with w equal to the model width d, and adds the two rows, each multiplied by a hashed ± 1 sign vector so that colliding n-grams interfere less:

$$e_t = \sum_n s_n \odot T[h_n(x_{t-n+1:t})]. \tag{1}$$

The vector e_t enters the residual stream through a learned gate before some of the layers. The table starts at zero, so a fresh model computes what it would without it, and it is trained sparsely: only the rows a batch touches change. The NanoGPT speedrun uses such a table (Jordan et al., 2024), and the idea of hashing features into shared rows is older (Weinberger et al., 2009; Svenstrup et al., 2017).

2.2 Multi-head addressing

Two n-grams in the same flat row are indistinguishable. Multi-head addressing hashes each n-gram with K independent hash functions per order, each into its own table of $P_{n,k}$ rows of width r, and decodes the concatenation with a learned projection:

$$e_t = W_V\big[\, s_{n,k} \odot T[\text{base}_{n,k} + u_{n,k}(x) \bmod P_{n,k}] \,\big]_{n,k}, \qquad W_V \in \mathbb{R}^{d \times 2Kr}. \tag{2}$$

Collisions become partial: two n-grams share a row in one head but rarely in all. Over-encoding splits the input embedding of n-gram tokens into heads in this way (Huang et al., 2025), and Engram uses $K = 8$ heads per order with tables of distinct prime sizes, together with a context-aware gate $\alpha_t = \sigma(\text{RMSNorm}(h_t)^\top \text{RMSNorm}(k_t)/\sqrt{d})$ on the memory output (Cheng et al., 2026). We call the addressing *Engram-style* and test its gate separately.

2.3 Reading a table from flash

A table of billions of parameters does not stay in the RAM of a phone or a laptop next to the model. Storing it in flash and reading what each token needs is the alternative (Alizadeh et al., 2024; Yu et al., 2025). Flash is read in pages, here 4 KiB; a random read takes tens of microseconds, and throughput is limited by the reads the drive can serve per second. At four bits per value with one 16-bit scale per row, a 64-wide head row takes 34 bytes, so a page holds 120 head rows. A flat table reads one page per order, two per token. Engram-style addressing reads one page per head, sixteen per token at $K = 8$, because the head rows of an n-gram lie in different tables. A RAM page cache keeps recently read pages, but under Engram-style addressing each cached page serves one head of one n-gram at a time.

3 PageGram

3.1 Layout

PageGram keeps the K heads per order of multi-head addressing and changes where their rows live. The table of each order is divided into P_n pages, and each page holds $K \cdot S$ head rows: S slots for each head. An n-gram selects its page with one hash and each head selects a slot inside that page with its own:

$$\text{row}_{n,k}(x) = \text{base}_n + (p_n(x)\,K + k)\,S + (u_{n,k}(x) \bmod S), \qquad p_n(x) = b_n(x) \bmod P_n, \tag{3}$$

where b_n is the n-gram's page hash and $u_{n,k}$ the hash of head k. All K head rows of an order lie in one page, so a token reads two pages. Everything else is unchanged: the $2K$ head rows are multiplied by hashed signs, concatenated and decoded by W_V, and the result enters the residual stream through a scalar gate per injection layer, $x \leftarrow x + g_j\, e_t$.

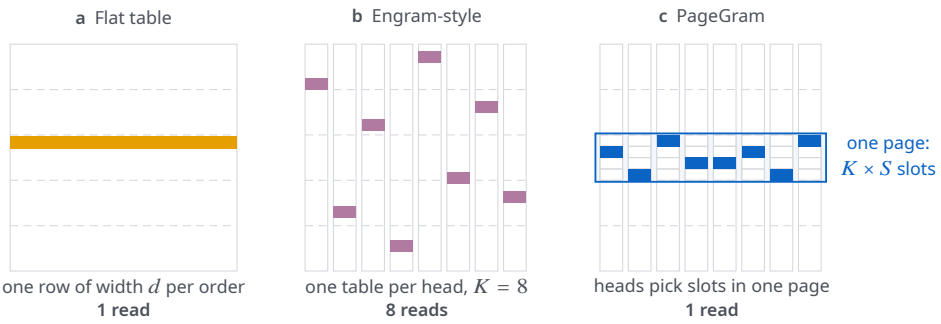


Figure 2 **Where an n-gram's rows live, for one n-gram order.** Dashed lines mark flash pages. **a** A flat table reads one wide row. **b** Engram-style addressing hashes each head into its own table, so the head rows lie in different pages. **c** PageGram hashes the n-gram to one page and lets each head pick one of its S slots there (here $S = 4$). With bigrams and trigrams a token reads 2, 16 and 2 pages.

3.2 What the slots control

For a fixed table size, every head of every layout has the same number of rows: $P_{n,k}$ for Engram-style addressing, $P_n S$ for PageGram. The rate at which an n-gram shares a row with others in one head is therefore the same. What S changes is how collisions in different heads relate. Two n-grams on the same page collide in each head with probability $1/S$, independently. Given a collision in one head, they share $(K-1)/S$ further heads on average. With Engram-style addressing, a second shared head is a coincidence of probability about $1/P_{n,k}$. S moves between two familiar designs. At $S = 1$ a page holds one row per head, and the n-grams on a page share all their rows, much as in a flat table. As S grows and the number of pages falls, the heads decouple, and at a single page PageGram becomes Engram-style addressing. Section 6 measures where in between learning works best.

3.3 Training and storage

The table starts at zero and is trained sparsely with a row-wise RMS update. Each row keeps one running mean of its squared gradient, the steps it misses are replayed as decay, and it moves by $\eta_T \, g/\sqrt{\hat{v}}$, the rule of the speedrun's n-gram table (Jordan et al., 2024). The table learning rate η_T is 70 times the dense learning rate, the best of the three values we tried for every table (Appendix B). W_V starts orthogonal with gain $\sqrt{2d/(2Kr)}$. Neither W_V nor the gates draw from the random stream that initialises the rest of the model, so every arm of a seed starts from the same core and computes the same function at step zero.

For deployment we store each 64-wide head row in NF4 (Dettmers et al., 2023), with one FP16 scale: 34 bytes. At $S = 15$ a page holds $8 \cdot 15 = 120$ head rows, 4,080 bytes, and fills a 4 KiB block. At $S = 4$ a page is 1,088 bytes, and three pages share a block. That makes the file 25% larger, but a token still reads two blocks. The lookup is cheap. Addresses come from integer hashes of the token ids, so they are known as soon as the tokens are, and reads can be issued before the model needs the rows.

4 Experimental setup

Model. A decoder with six pre-norm blocks of width $d = 384$, six attention heads of 64 with rotary position embeddings (Su et al., 2021), RMSNorm (Zhang and Sennrich, 2019), a GELU MLP of width $4d$ and untied input and output embeddings over a vocabulary of 65,536 tokens: 61M parameters, 10.6M of them in the blocks. The memory vector enters the residual stream before blocks 0 and 4. The model is small on purpose. A small model can store little of the data's local statistics in its own weights, so the table has room to help, and its runs are short enough for five seeds of every arm.

Tables. Every table holds 200M parameters, within 0.1%. The flat table has 520,832 rows of width 384, half for bigrams and half for trigrams. Engram-style addressing uses $K = 8$ heads per order and sixteen tables of distinct prime sizes near 195,300 rows of width 64. PageGram uses the same heads

Table 1 Held-out loss after 60M training tokens, mean and standard deviation over five seeds; the deltas are paired by seed (every arm of a seed shares the core initialisation and the data order). *Wins*: seeds on which the arm is below the flat table. *4-bit*: loss change with the table snapped to NF4. *Reads*: random flash reads per token for a deployed table. Throughput: two runs sharing one RTX 5090.

Memory	Params	Reads	Held-out loss	Δ no table	Δ flat	Wins	4-bit	Tok/s
No table	—	0	5.484 ±0.025	—	+0.333 ±0.063	0/5	—	113K
Flat table	200M	2	5.151 ±0.043	−0.333 ±0.063	—	—	+0.0006	110K
Engram-style	200M	16	5.225 ±0.075	−0.259 ±0.094	+0.074 ±0.033	0/5	+0.0004	109K
PageGram, S=4	200M	2	**5.084 ±0.052**	−0.400 **±0.045**	−0.067 **±0.063**	5/5	+0.0004	108K
PageGram, S=15	200M	2	5.126 ±0.088	−0.357 ±0.100	−0.025 ±0.092	3/5	+0.0004	109K

and row width with $S \in \{1, 4, 8, 15, 60\}$ and a prime number of pages per order. At $S = 4$ that is 48,823 and 48,821 pages. Both multi-head tables are decoded by a W_V of 0.39M parameters. Hashes, signs and the page hash are those of our training framework, unchanged; Appendix A lists every setting.

Data and training. A sample of 228M tokens of a multilingual pretraining mixture: web text, reference and educational text, code and mathematics, and a small share of dialogue, in English, Chinese, German, Japanese and Russian. Each run trains for 457 steps of 131,072 tokens, 60M tokens, read as random windows of 1,024 tokens. Training uses AdamW (Loshchilov and Hutter, 2019) (learning rate $2 \cdot 10^{-3}$, $\beta = (0.9, 0.95)$, weight decay 0.1 on matrices) and a warmup–stable–decay schedule with 5% warmup and a linear decay to zero over the last 20% (Hu et al., 2024; Hägele et al., 2024). The held-out loss is measured on the same 2.1M tokens from a separate held-out split for every run.

Protocol. All arms of a seed share the core initialisation and the order of the training windows. The table starts at zero, so each arm computes exactly what the table-free model computes at step zero, which the harness checks bit for bit. We run five seeds for the comparison of tables and for $S = 4$ and $S = 15$, three for the other settings, the ablations and the runs with twice the tokens, and one for the larger model. All runs used one RTX 5090, two at a time.

5 Quality at equal size

Table 1 and Fig. 1 compare the tables after 60M training tokens. Every table helps a model of this size a great deal at this point: a flat table lowers held-out loss by 0.333, Engram-style addressing by 0.259 and PageGram with $S = 4$ by 0.400. PageGram is lower than the flat table on all five seeds, by 0.067 on average. A paired t-test gives $p = 0.08$, and the sign test, five of five, gives a one-sided $p = 0.03$. It is lower than Engram-style addressing on all five seeds, by 0.142 ($p = 0.02$). Its lead appears early and holds: PageGram is ahead at the first evaluation, after 13M tokens, and at every one after (Fig. 1a).

After 60M tokens Engram-style addressing is *worse* than the flat table on every seed, by 0.074 ($p = 0.008$). Multi-head addressing was introduced to beat flat tables, and at larger scale it does (Huang et al., 2025); with twice the tokens it does here too. Early in training, with every head decoupled from the others, it learns more slowly than one shared row. Within the same multi-head family, then, the placement of the heads decides whether splitting a row into heads helps or hurts. Section 6 shows that the placement matters through collisions.

PageGram with $S = 15$, the page format that fills a 4 KiB block exactly, sits between the two on average, with one notable seed. Four of its five seeds end between 5.071 and 5.105, as good as $S = 4$; the fifth ends at 5.283, behind from the first evaluation. We return to this in Section 6.

Longer training and a larger model. Table 2 repeats the comparison with twice the tokens and, separately, with a larger model. With more training every table helps less: after 120M tokens the best table is 0.095 below no table, against 0.40 after 60M. A model of this size learns much of the local statistics a table supplies once it has seen enough text, and the table's early contribution shrinks accordingly. The order

Table 2 Held-out loss at two training lengths and for a larger model, mean over the seeds run. The last column is paired by seed, mean and standard deviation.

Setting	Seeds	No table	Flat	Engram-style	PageGram S=4	PageGram − flat
60M tokens	5	5.484	5.151	5.225	5.084	−0.067 ±0.063
120M tokens	3	4.220	4.159	4.125	4.136	−0.024 ±0.052
Larger model, 60M	1	5.567	5.417	—	5.473	+0.056

Table 3 Collision statistics of the two multi-head layouts at 200M parameters on 20M held-out tokens, per head (all eight heads; the range is over heads). λ: distinct n-grams per head-row. Var/mean: the variance of the per-row count over its mean (1 for ideal random hashing). *Foreign*: the share of a row's occurrences that belong to other n-grams, as training sees it. The last two columns take n-gram pairs that collide in head 0 and count how many of the other seven heads they also share.

Order	Layout	λ	Var/mean	Foreign	Further heads shared	Any further
Bigram	Engram-style	25.3	0.993–1.003	0.669	<0.001	<0.01%
Bigram	PageGram (S=15)	25.3	0.996–1.005	0.669	0.467	38.3%
Trigram	Engram-style	61.7	0.993–1.005	0.894	<0.001	<0.01%
Trigram	PageGram (S=15)	61.7	0.995–1.002	0.894	0.467	38.3%

of the layouts changes too. Engram-style addressing now beats the flat table on all three seeds, by −0.034 on average, and PageGram is level with it, +0.010 apart. PageGram stays below the flat table on average (−0.024, lower on 2/3 seeds, with individual seeds between −0.061 to +0.035). In the larger model ($d = 512$, 8 layers) after 60M tokens, PageGram ends 0.056 behind the flat table on one seed, within the spread of paired differences in the small model. Early in training correlated collisions learn fastest; later the multi-head layouts converge, and PageGram keeps Engram-style addressing's quality while reading eight times less (Section 8).

The deployed format costs nothing measurable. Snapping any of the tables to NF4 with one scale per 64 values raises the held-out loss by 0.0004 to 0.0006. Training with a table runs within 5% of the speed of the table-free model in our implementation, where the lookup runs eagerly next to the compiled network.

6 Correlated collisions

The same collisions, differently arranged. Table 3 measures the two multi-head layouts on 20M held-out tokens, replaying the exact addresses the model uses. Per head they cannot be told apart. Each head row holds 25.3 distinct bigrams or 61.7 distinct trigrams. The spread of that count over rows matches ideal random hashing, and the share of a row's occurrences that belongs to other n-grams, the interference training sees, is 0.669 for bigrams and 0.894 for trigrams in both layouts. The only difference is across heads. Two n-grams that collide in one PageGram head share another 0.47 heads on average, exactly $(K-1)/S$ at $S = 15$, while under Engram-style addressing the count is below 0.001. The hashing is sound in both; the layouts differ in correlation alone.

Fewer, closer neighbours. The correlation changes whom an n-gram shares rows with. With λ n-grams per head row, an n-gram meets about $K\lambda$ head-level collisions in every layout. Engram-style addressing spreads them over $K\lambda$ distinct n-grams, one shared head each. PageGram concentrates them on its page-mates: of the λS n-grams on its page, a fraction $1-(1-1/S)^K$ shares at least one head with it. Relative to Engram-style addressing, an n-gram then has 0.79 times as many distinct neighbours at $S = 15$, 0.45 times at $S = 4$ and 0.13 times at $S = 1$, and correspondingly more shared heads with each. At $S = 1$ every neighbour shares all eight heads, and the n-gram is as identifiable as in a flat table.

Table 4 The slots per head per page, S, at a fixed 200M-parameter table, after 60M training tokens. *Shared*: the expected number of further heads in which two n-grams that collide in one head also collide, $(K-1)/S$; S=1 makes collisions total, as in a flat table, and $S \to \infty$ is Engram-style addressing. Deltas against the flat table are paired by seed.

S	Shared	Seeds	Held-out loss	Δ flat	Wins
1	7.00	3	5.140 ±0.041	−0.023 ±0.030	3/3
4	1.75	5	5.084 ±0.052	−0.067 ±0.063	5/5
8	0.88	3	5.199 ±0.074	+0.036 ±0.075	1/3
15	0.47	5	5.126 ±0.088	−0.025 ±0.092	3/5
60	0.12	3	5.171 ±0.103	+0.008 ±0.099	2/3
∞ (Engram-style)	0.00	5	5.225 ±0.075	+0.074 ±0.033	0/5

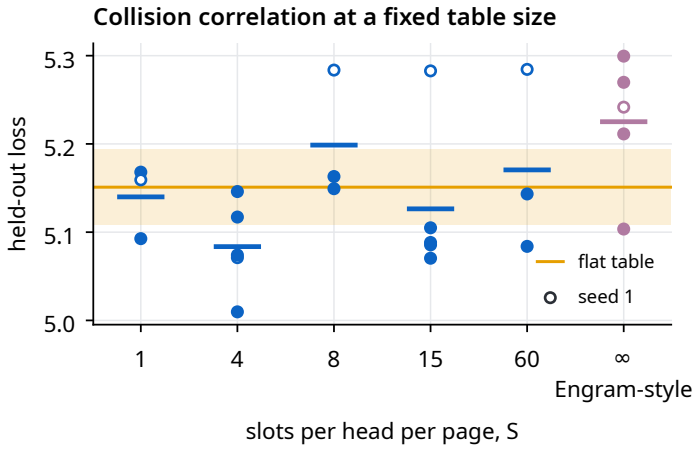


Figure 3 **The slots per page set how collisions correlate.** Final held-out loss after 60M tokens against S at a fixed table size of 200M parameters; dots are seeds (hollow: the seed discussed under Stability), bars means. $S = 1$ behaves like a flat table (band: flat table, mean and standard deviation over five seeds) and $S \to \infty$ is Engram-style addressing.

The curve. Table 4 and Fig. 3 vary S at a fixed table size, after 60M training tokens, where the layouts differ most. $S = 1$ lands close to the flat table, as the analysis predicts: 0.023 below it on each of its three seeds. $S = 4$ is best, 0.067 below the flat table on every seed. Larger S is worse on average, and the limit, Engram-style addressing, is worst of all. Learning prefers collisions that are partial, as multi-head addressing intends, *and* correlated, which hashing designed for independence avoids. We read the curve as a balance of two forces. Larger S makes an n-gram's code more distinct. Smaller S concentrates its interference on a few page-mates, whose shared rows the decoder can learn to separate together, instead of on hundreds of unrelated n-grams with one row each.

Stability. The averages hide a sharper pattern. On one seed of the five, every layout with $S \geq 8$ falls behind within the first 13M tokens and ends at the same loss: 5.284 at $S = 8$, 5.283 at $S = 15$ and 5.285 at $S = 60$, worse than Engram-style addressing on that seed (5.242). The table does not collapse in these runs: every row is trained and its scale matches the other runs. On the same seed, with the same initialisation and data, $S = 1$ and $S = 4$ are unaffected. Strongly correlated layouts, with at least 1.75 further shared heads per collision, get the advantage on every seed. Weakly correlated layouts get it on some seeds and lose it on others, and in this study the loss happened from the start. We therefore recommend $S = 4$, the layout that combines the best mean with no failed seed. Its mean is not separable from that of $S = 15$ with five seeds ($p = 0.45$); its reliability is.

Table 5 Ablations at 60M tokens; deltas are paired by seed against the reference above them.

Arm	Seeds	Held-out loss	Δ reference
PageGram S=4	5	5.084 ±0.052	reference
+ Engram context gate	3	5.206 ±0.044	+0.140 ±0.016
a quarter of the table (50M)	3	5.272 ±0.035	+0.206 ±0.086
PageGram S=15	5	5.126 ±0.088	reference
+ Engram context gate	3	5.221 ±0.041	+0.062 ±0.131
a quarter of the table (50M)	3	5.324 ±0.056	+0.166 ±0.063

Table 6 Reading deployed 4-bit tables from flash, on 20M held-out tokens. *File*: the table in 4 KiB blocks; at S=4 three 1,088-byte pages share a block. *Reads*: distinct blocks per token without a cache. Then the reads per token that miss an LRU page cache of the given size.

				Reads per token with a page cache of			
Table	Layout	File (GiB)	Reads	64 MiB	256 MiB	1 GiB	4 GiB
3.2B	PageGram S=4	1.99	2	1.29	1.06	0.50	0.03
3.2B	PageGram S=15	1.59	2	1.28	1.02	0.36	0.02
3.2B	Engram-style	1.59	16	11.35	9.26	3.59	0.02
10B	PageGram S=4	6.21	2	1.32	1.17	0.89	0.32
10B	PageGram S=15	4.97	2	1.32	1.16	0.85	0.18
10B	Engram-style	4.97	16	11.67	10.48	8.10	1.54
20B	PageGram S=4	12.42	2	1.33	1.20	0.99	0.63
20B	PageGram S=15	9.93	2	1.33	1.19	0.96	0.55
20B	Engram-style	9.93	16	11.75	10.77	9.18	5.27

Lookups per second from NVMe, 20B table	1 in flight	2 in flight	16 in flight	64 in flight
PageGram	8,343	23,615	33,508	118,874
Engram-style	1,048	4,607	5,205	17,405

7 Ablations

Engram's context gate. Engram scales the memory by a gate that compares the hidden state with a key computed from the memory output, and its ablations find the gate important at their scale (Cheng et al., 2026). We add the same gate to PageGram, with one key projection shared by the injection layers and a query norm for each (Table 5). At $S = 4$ it raises the held-out loss by 0.140, on all three seeds (0.146, 0.152 and 0.121). At $S = 15$ it does not help either, although the failed seed of $S = 15$ makes that comparison noisy. At the size of our study the scalar gate is the better choice, and every other result uses it.

Table size. A quarter of the table, 50M parameters at $S = 4$, keeps about half of the full table's gain over no table: 0.196 of 0.400, between a third and two thirds depending on the seed. Size still matters more than layout. Quartering the table costs 0.206, three times what PageGram gains over a flat table of full size, and the quarter-size PageGram table ends 0.109 behind the 200M flat table.

4-bit storage. Snapping a trained table to NF4 with one FP16 scale per 64 values changes the held-out loss by +0.0004 to +0.0006 for every table and every size we trained. The deployed format of Section 8 costs no quality.

8 Reading from flash

We replay 20M held-out tokens through deployed 4-bit tables of three sizes and count the 4 KiB blocks each token reads (Table 6 and Fig. 4). Without a cache the counts follow from the layouts: two blocks

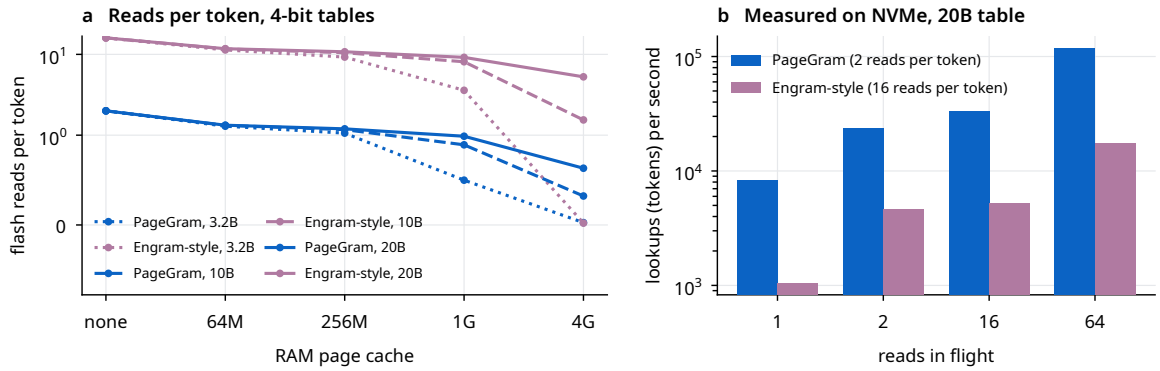


Figure 4 **Flash cost of a deployed table.** **a** Random reads per token that miss an LRU page cache in RAM, for 4-bit tables of 3.2B, 10B and 20B parameters, on 20M held-out tokens. PageGram at $S = 4$ against Engram-style addressing; the vertical axis is linear below one read and logarithmic above. **b** Lookups per second from an NVMe drive with direct 4 KiB reads of a 20B table, against the number of reads in flight.

per token for PageGram and the flat table, sixteen for Engram-style addressing. A page cache in RAM narrows the gap less than one might hope. On the 20B table with 1 GiB of cache, PageGram needs 0.99 reads per token and Engram-style addressing 9.18; with 4 GiB, 0.63 and 5.27. At every size, and every cache well below the size of the table, PageGram reads eight to nine times less. A cached PageGram page serves every head of every n-gram on it, while a cached Engram-style page serves one head at a time. When the cache approaches the size of the table, both layouts converge, as they must: a 4 GiB cache holds the whole 3.2B table.

The read counts carry over to real throughput. On the NVMe drive of our training machine, with direct reads that bypass the operating system's cache, a 20B PageGram table serves 8,343 lookups per second with one read in flight against 1,048 for Engram-style addressing, eight times as many, and 118,874 against 17,405 with 64 in flight. Across depths from one to 64 the ratio is five to eight. Generating text needs few lookups per second; reading a prompt needs one per prompt token at once, and that is where the difference lands. $S = 4$ costs storage, not reads: its 1,088-byte pages fit three to a block, which makes the file 25% larger than at $S = 15$ (12.4 against 9.9 GiB at 20B), while a token still reads two blocks and the cache behaves almost as well.

9 Related work

N-gram memory. N-Grammer adds latent n-gram embeddings to a Transformer (Roy et al., 2022). Over-encoding scales hashed n-gram input embeddings and finds a log-linear relation between their size and the training loss (Huang et al., 2025), and Engram turns them into a conditional memory with K hash heads per order, prime-sized tables and a context-aware gate, scaled to 27B parameters (Cheng et al., 2026). Liu et al. (2026) find that scaling embeddings can beat scaling experts. SCONE keeps n-gram embeddings in off-accelerator memory to cut inference compute (Yu et al., 2025), and the NanoGPT speedrun trains a flat hashed n-gram table with a row-wise update (Jordan et al., 2024), the rule we use. PageGram keeps Engram's heads and changes only where their rows are stored; in our measurements that one change cuts flash reads eightfold at no cost in quality.

Hashing and locality. Sharing parameters through hashing goes back to feature hashing (Weinberger et al., 2009) and hash embeddings (Svenstrup et al., 2017). Using several independent hashes to make collisions partial is the idea behind Bloom filters (Bloom, 1970) and count-min sketches (Cormode and Muthukrishnan, 2005). Blocked Bloom filters place all of a key's hash bits in one cache line for locality and accept a small loss of accuracy for it (Putze et al., 2007). PageGram applies the same blocking to learned tables, where the correlation it introduces costs no quality and helps early in training.

Memory layers and on-device inference. Product-key memory layers add a large trainable memory read by attention-like lookups (Lample et al., 2019; Berges et al., 2024); their addresses depend on the hidden state, whereas an n-gram table's depend only on the tokens, which lets reads be issued before the model runs. Running models whose weights exceed the device's memory from flash is studied by Alizadeh et al. (2024). NF4 is the 4-bit format of QLoRA (Dettmers et al., 2023).

10 Conclusion

Multi-head n-gram tables decouple their heads to make collisions partial, and pay for it with one random read per head. PageGram keeps the heads but puts them in one page. Each n-gram then costs two reads per token, as in a flat table, and collisions become correlated across heads. That correlation costs no quality. At equal size PageGram beats a flat table and Engram-style addressing on every seed early in training. Later it stays level with Engram-style addressing and ahead of the flat table on average. A single parameter, the slots per head per page, controls how much correlation there is, and four slots per head per page gave the best and most stable results. Stored at four bits, a PageGram table can be served from flash with eight to nine times fewer reads than an Engram-style table, at no measurable cost in quality. PageGram is the n-gram memory of our current models.

References

Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. In *Annual Meeting of the Association for Computational Linguistics*, 2024.

Vincent-Pierre Berges, Barlas Oğuz, Daniel Haziza, Wen-tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory layers at scale. *arXiv preprint arXiv:2412.09764*, 2024.

Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

Xin Cheng, Rui Tian, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, et al. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*, 2026.

Graham Cormode and S. Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro Von Werra, and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. In *Advances in Neural Information Processing Systems*, 2024.

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.

Hongzhi Huang, Defa Zhu, Banggu Wu, Yutao Zeng, Ya Wang, Qiyang Min, and Xun Zhou. Over-tokenized transformer: Vocabulary is generally worth scaling. In *International Conference on Machine Learning*, 2025.

Keller Jordan, Jeremy Bernstein, Brendan Rappazzo, et al. modded-nanogpt: Speedrunning the NanoGPT baseline. `https://github.com/KellerJordan/modded-nanogpt`, 2024.

Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, 2019.

Hong Liu, Jiaqi Zhang, Chao Wang, Xing Hu, Linkun Lyu, Jiaqi Sun, et al. Scaling embeddings outperforms scaling experts in language models. *arXiv preprint arXiv:2601.21204*, 2026.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Felix Putze, Peter Sanders, and Johannes Singler. Cache-, hash- and space-efficient bloom filters. In *Experimental Algorithms (WEA)*, volume 4525 of *Lecture Notes in Computer Science*, 2007.

Aurko Roy, Rohan Anil, Guangda Lai, Benjamin Lee, et al. N-Grammer: Augmenting transformers with latent n-grams. *arXiv preprint arXiv:2207.06366*, 2022.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Dan Tito Svenstrup, Jonas Meinertz Hansen, and Ole Winther. Hash embeddings for efficient word representations. In *Advances in Neural Information Processing Systems*, 2017.

Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *International Conference on Machine Learning*, 2009.

Da Yu, Edith Cohen, Badih Ghazi, Yangsibo Huang, Pritish Kamath, Ravi Kumar, Daogao Liu, and Chiyuan Zhang. Scaling embedding layers in language models. In *Advances in Neural Information Processing Systems*, 2025.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *arXiv preprint arXiv:1910.07467*, 2019.

A Setup details

Table 7 Settings shared by every run.

Model	
Blocks, width	6 pre-norm blocks, $d = 384$, RMSNorm
Attention	6 heads of 64, rotary positions, causal, context 1,024
MLP	GELU, width $4d$
Embeddings	untied, vocabulary 65,536; 61.0M parameters in all, 10.6M in the blocks
Memory injection	before blocks 0 and 4, scalar gate per injection layer (init $1/2$)
Tables (200M parameters each)	
Flat	520,832 rows × 384, bigram and trigram halves, hashed signs
Engram-style	16 tables (8 heads × 2 orders), distinct primes near 195,300 rows × 64
PageGram	8 heads × S slots per page, prime page counts per order, rows × 64
Decoder	$W_V \in \mathbb{R}^{384\times1024}$ for the multi-head tables, orthogonal init, gain $\sqrt{2d/(2Kr)}$
Training	
Tokens	457 steps × 131,072 tokens (60M), random windows of 1,024
Longer runs	915 steps (120M tokens), evaluated every 200 steps
Larger model	$d = 512$, 8 blocks, 8 heads of 64 (92M parameters), otherwise unchanged
Dense optimiser	AdamW, learning rate $2 \cdot 10^{-3}$, $\beta = (0.9, 0.95)$, weight decay 0.1 on matrices
Schedule	5% linear warmup, constant, linear decay to 0 over the last 20%
Table optimiser	row-wise RMS, $\beta_2 = 0.95$, learning rate 70× the dense one, same schedule
Clipping, precision	gradient norm 1.0 (dense parameters), BF16 autocast, FP32 table
Evaluation	
Held-out loss	2,048 fixed windows (2.1M tokens) of a separate held-out split
Hardware	one RTX 5090, two runs at a time

B Table learning rate

We set the table learning rate by a short probe for each table type (Table 8). ×70, the multiplier of the speedrun's n-gram table, is best for all three, so every arm uses it. Engram trains its tables with Adam at five times the dense learning rate (Cheng et al., 2026); with a row-normalised update the multipliers are not comparable.

C Every run

D Flash measurements

The simulator replays a token stream through the addressing of the model, written in C and checked token by token against the model's own addresses. It counts the distinct 4 KiB blocks each token reads and passes them through an LRU cache of the given size. Head rows take 34 bytes (64 NF4 values and an FP16 scale), so a block holds 120 rows of an Engram-style table or one PageGram page at $S = 15$, or three pages at $S = 4$. For Engram-style addressing each head's table is stored contiguously. The throughput measurement writes a 9.9 GiB file, drops it from the page cache and reads the blocks of 20,000 to 400,000 tokens with `O_DIRECT` from 1 to 64 threads. It reports lookups per second, and a mean read latency of 60 μs with one read in flight.

E Verification

Before the study, the harness passed all 20 of its self-checks. The flat, Engram-style and PageGram tables hold the same number of parameters. Every address lies in its table and region, every head row of a PageGram n-gram lies in its page, and no n-gram reaches across a document boundary. With the table at zero, every arm computes the table-free model's loss bit for bit, and every arm of a seed shares its core initialisation. The sparse table gradient equals a dense reference, the row-wise update and

Table 8 Table learning rate, as a multiple of the dense learning rate: held-out loss after 30M tokens, seed 0. ×70 is best for all three tables and is used throughout.

Table	×20	×70	×200
Flat table	6.282	**6.148**	6.297
Engram-style	6.244	**6.180**	6.193
PageGram S=15	6.236	**6.160**	6.162

Table 9 Every run's final held-out loss, by seed: 60M tokens unless marked.

Arm	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4
No table	5.455	5.471	5.480	5.494	5.520
Flat table	5.175	5.164	5.150	5.188	5.078
Engram-style	5.270	5.242	5.211	5.299	5.104
PageGram S=1	5.168	5.159	5.093	—	—
PageGram S=4	5.010	5.071	5.117	5.146	5.074
PageGram S=8	5.149	5.284	5.163	—	—
PageGram S=15	5.105	5.283	5.088	5.070	5.086
PageGram S=60	5.143	5.284	5.084	—	—
S=4 + context gate	5.156	5.223	5.239	—	—
S=4, quarter table	5.313	5.249	5.255	—	—
S=15 + context gate	5.268	5.197	5.198	—	—
S=15, quarter table	5.273	5.384	5.316	—	—
120M tokens					
No table	4.231	4.154	4.276	—	—
Flat table	4.146	4.157	4.175	—	—
Engram-style	4.139	4.114	4.124	—	—
PageGram S=4	4.182	4.112	4.114	—	—
Larger model (d=512, 8 layers), 60M tokens					
No table	5.567	—	—	—	—
Flat table	5.417	—	—	—	—
PageGram S=4	5.473	—	—	—	—

its replayed decay match a hand-computed rule, and the 4-bit format keeps zero rows at zero with the expected maximum error. The C simulator's block addresses equal the model's for every token. Finally, every arm trains, evaluates and writes its summary on the GPU.